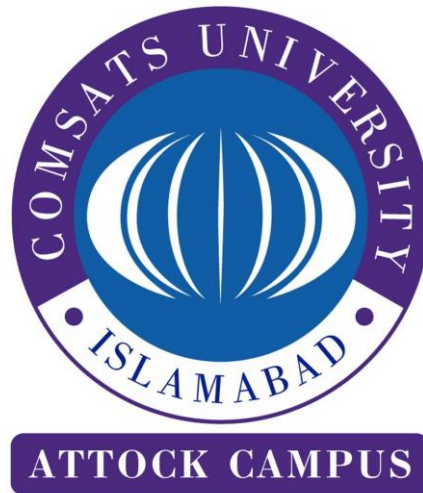


Data Structures Lab

Assignment 01



By

Mamoona Anwar

FA25-BCS-020

Submitted to,

Mr. Bilal Bukhari

September 16, 2026

Department of Computer Science
COMSATS University Islamabad, Attock Campus

Part 01:

Code:

```
#include <iostream>

#include <cmath> // For abs() function used in Part D


using namespace std;


const int CAPACITY = 20;


struct ArrayList {
    int data[CAPACITY];
    int size = 0;
};


// Global list as per starter code
ArrayList list;


// Function Prototypes
bool insertEnd(ArrayList &list, int value);
bool insertAtBeginning(ArrayList &list, int value);
bool deleteAtPosition(ArrayList &list, int position);
void displayList(const ArrayList &list);


int main() {
    // --- Part A: Create and Populate the ArrayList ---
```

```

cout << "--- Part A: Create and Populate the ArrayList ---" << endl;

// Replace '08' with the last two digits of your own registration number
int reg_no_last_two = 8;

int initial_values[] = {18, 7, 45, 11, 36, reg_no_last_two, 21, 13, 29};

for (int i = 0; i < 9; i++) {
    insertEnd(list, initial_values[i]);
}

cout << "Initial ArrayList: ";
displayList(list);
cout << endl;

// --- Part B: Pointer Traversal, Sum, Minimum and Maximum ---
cout << "--- Part B: Traversal, Min, Max, Sum ---" << endl;
int *ptr = list.data;
int *minPtr = list.data;
int *maxPtr = list.data;
int sum = 0;

// Traverse using pointer arithmetic
for (int i = 0; i < list.size; i++) {
    sum += *ptr; // Calculate sum

```

```

    if (*ptr < *minPtr) {
        minPtr = ptr; // Point to minimum
    }
    if (*ptr > *maxPtr) {
        maxPtr = ptr; // Point to maximum
    }
    ptr++; // Move to next element
}

cout << "Minimum Value: " << *minPtr << endl;
cout << "Maximum Value: " << *maxPtr << endl;
cout << "Sum: " << sum << endl;
cout << endl;

// --- Part C: Find the Median ---
cout << "--- Part C: Find the Median ---" << endl;

// Create temporary array to sort without rearranging original list
int tempArr[CAPACITY];
for (int i = 0; i < list.size; i++) {
    tempArr[i] = list.data[i];
}

// Manual sorting (Bubble Sort) as STL sort is not allowed
for (int i = 0; i < list.size - 1; i++) {

```

```

    for (int j = 0; j < list.size - i - 1; j++) {
        if (tempArr[j] > tempArr[j + 1]) {
            int temp = tempArr[j];
            tempArr[j] = tempArr[j + 1];
            tempArr[j + 1] = temp;
        }
    }
}

// Find middle value (size is 9, so index 4 is the middle)
int medianValue = tempArr[list.size / 2];
int *medianPtr = list.data; // Reset to start to find first occurrence

for (int i = 0; i < list.size; i++) {
    if (list.data[i] == medianValue) {
        medianPtr = &list.data[i];
        break; // Stop at first occurrence
    }
}

cout << "Median Value: " << *medianPtr << endl;
cout << endl;

// --- Part D: Two Averages and Closest Value ---
cout << "--- Part D: Averages and Closest Value ---" << endl;
double generalAverage = (double)sum / list.size;

```

```
double specialAverage = (*minPtr + *medianPtr + *maxPtr) / 3.0;
```

```
int *closestPtr = list.data;
```

```
int closestPosition = 0;
```

```
double minDistance = abs(list.data[0] - specialAverage);
```

```
ptr = list.data; // Reset pointer to start
```

```
for (int i = 0; i < list.size; i++) {
```

```
    double currentDistance = abs(*ptr - specialAverage);
```

```
    // If we find a strictly smaller distance, update closest pointer
```

```
    if (currentDistance < minDistance) {
```

```
        minDistance = currentDistance;
```

```
        closestPtr = ptr;
```

```
    }
```

```
    ptr++;
```

```
}
```

```
closestPosition = closestPtr - list.data; // Pointer arithmetic for position
```

```
cout << "General Average: " << generalAverage << endl;
```

```
cout << "Special Average: " << specialAverage << endl;
```

```
cout << "Closest Value: " << *closestPtr << endl;
```

```
cout << "Position of Closest Value: " << closestPosition << endl;
```

```
cout << endl;

// --- Part E: Final Calculations and ArrayList Modification ---

cout << "--- Part E: Final Calculations & Modification ---" << endl;

double averageDifference = abs(generalAverage - specialAverage);

double finalScore = abs(*closestPtr - generalAverage) + abs(*closestPtr - specialAverage) +
averageDifference;

cout << "Difference Between Averages: " << averageDifference << endl;

cout << "Final Score: " << finalScore << endl;

// Delete the element at closestPosition

deleteAtPosition(list, closestPosition);

cout << "ArrayList After Deletion: ";

displayList(list);

// Round specialAverage to nearest integer and insert at beginning

int roundedSpecialAvg = (int)(specialAverage + 0.5); // Simple rounding for positive numbers

insertAtBeginning(list, roundedSpecialAvg);

cout << "Final ArrayList After Insertion: ";

displayList(list);

return 0;

}
```

```
// --- Function Implementations ---
```

```
bool insertEnd(ArrayList &list, int value) {  
    if (list.size >= CAPACITY) {  
        return false; // List is full  
    }  
    list.data[list.size] = value;  
    list.size++;  
    return true;  
}
```

```
bool insertAtBeginning(ArrayList &list, int value) {  
    if (list.size >= CAPACITY) {  
        return false; // List is full  
    }  
    // Shift elements to the right  
    for (int i = list.size; i > 0; i--) {  
        list.data[i] = list.data[i - 1];  
    }  
    list.data[0] = value;  
    list.size++;  
    return true;  
}
```

```
bool deleteAtPosition(ArrayList &list, int position) {
```

```
if (position < 0 || position >= list.size) {  
    return false; // Invalid position  
}  
  
// Shift elements to the left to fill the gap  
for (int i = position; i < list.size - 1; i++) {  
    list.data[i] = list.data[i + 1];  
}  
list.size--;  
return true;  
}
```

```
void displayList(const ArrayList &list) {  
    cout << "[ ";  
    for (int i = 0; i < list.size; i++) {  
        cout << list.data[i];  
        if (i < list.size - 1) {  
            cout << ", ";  
        }  
    }  
    cout << "]" << endl;  
}
```

Output:

```
C:\Users\LENOVO\Desktop\Lab Assignment 01 DS.exe
--- Part A: Create and Populate the ArrayList ---
Initial ArrayList: [ 18, 7, 45, 11, 36, 8, 21, 13, 29 ]

--- Part B: Traversal, Min, Max, Sum ---
Minimum Value: 7
Maximum Value: 45
Sum: 188

--- Part C: Find the Median ---
Median Value: 18

--- Part D: Averages and Closest Value ---
General Average: 20.8889
Special Average: 23.3333
Closest Value: 21
Position of Closest Value: 6

--- Part E: Final Calculations & Modification ---
Difference Between Averages: 2.44444
Final Score: 4.88889
ArrayList After Deletion: [ 18, 7, 45, 11, 36, 8, 13, 29 ]
Final ArrayList After Insertion: [ 23, 18, 7, 45, 11, 36, 8, 13, 29 ]

-----
Process exited after 0.2711 seconds with return value 0
Press any key to continue . . .
```